

Nevo Test Drive Service

The objective is to build a service that will provide on-demand scheduling of EV test drives to customers. This document details both the functional requirements, and our method of scoring. Below you can see the assignment, and at the bottom, you can see how you will be graded. We **strongly** recommend that you read the entire document before implementing.

Part 1: Backend Service

The backend service will provide 2 functions:

A: Request the availability of a vehicle for a given datetime and location

- Vehicle of the given type must be available on the requested day at the requested location
- Vehicle must not already be scheduled for a test drive at those dates
- Test drive requests must be distributed evenly across available vehicles

Request Example

```
location: Dublin
vehicleType: Volkswagen_ID4
startDateTime: 2023-11-01T09:00:00Z
durationMins: 45
```

B: Schedule a test drive for a vehicle and customer

- Vehicle slot will be reserved if available

Request Example

```
vehicleId: 017
startDateTime: 2023-11-01T09:00:00Z
durationMins: 45
customerName: John Smith
customerPhone: +353851234567
customerEmail: John@Smith.com
```

Part 2: Front-End UI

Provide a simple front-end UI component that uses the above services to allow a user to book a test drive.

- Vehicle type is configurable in the component when embedding on the page (i.e. the user doesn't need to select it)
- Bookings are available for up to 14 days in the future

Implementation Notes

- Third-party packages can be used where appropriate.
- Example vehicle details are provided as a JSON file, data persistence must use a database of some kind. You are welcome to use either NoSQL or SQL depending on which you think is more appropriate.
- Timezones can remain as UTC
- You do not need to host the services or front-end, but please provide instructions on how we can test your solution locally, and include a .md file indicating how you might host it (e.g. EC2 vs ECS or on some other provider)

AI Usage

You are welcome to use any AI as much as you want, however you will be expected to explain any and all details about your code live to us. So naturally, if you vibe code the entire project that will not be conducive to passing.

Scoring

Scoring is broken down into 2 categories.

1. **Binary** - You need to hit these marks or you will not make it to the next round
2. **Numerical** - You will be graded on a scale here.

Binary

1. The core of your solution must be functional and bug free. If the UI flickers in some place that's not a big deal but if the core booking algorithm doesn't meet the criteria, that will be a failing grade.
2. You must adequately solve for **all** race conditions
3. Your solution must have a `./dev.sh` script that allows us to stand it up and test it in one step. This script must not assume anything is pre-installed on the reviewer's system, and should handle anything that is already gracefully installed. e.g if docker is not installed, and you used docker, you should install docker.

Numerical

Rated out of 100

- **Architecture (30)** — Service structure, separation of concerns, API design, choice of communication patterns, and how well your system would scale or adapt to new requirements.
- **Even Distribution Algorithm (10)** — The quality and correctness of your approach to distributing test drive requests evenly across available vehicles. We're looking for a thoughtful strategy, not just random assignment.
- **User Experience (20)** — How intuitive and polished the front-end booking flow feels. Consider loading states, error feedback, responsiveness, and whether a real customer could comfortably use it.
- **Code Quality (10)** — Readability, consistent style, sensible naming, and appropriate use of language features. Code should be easy to follow without excessive comments.
- **Documentation (10)** — Clear setup instructions, explanation of key design decisions, and any trade-offs you made. We value concise, useful documentation over volume.
- **Automated Tests (10)** — Meaningful test coverage of critical paths, particularly around booking logic and race conditions. We care more about what you test than how many tests you have.
- **Narrated Video Demo (5)** — A short walkthrough demonstrating the booking flow and even distribution in action. See the Video Demo section below for specifics.
- **Git Hygiene (5)** — Clean commit history with meaningful messages that tell the story of how you built the solution. Logical, incremental commits over a single large one.

Video demo

As part of your submission, you need to submit a narrated video demo. It should be no longer than 8 minutes. You should do the following in the demo:

1. Book a model for a slot
2. Attempt to book the same model for the same slot (should indicate no availability)
3. Attempt to book a different available car for the same slot.
4. Attempt to book a model for a slot 15 mins after the previous booking.

In other words, you should present to us the even distribution algorithm in action, as well as the user experience.

Submission

You should respond via email within 7 days of receiving the assignment. In your submission, please include the following inside of a .zip file.

- A git repository, containing all of your work
- A video explainer of your solution, as explained above
- A journal.md where you write anything else you would like to include about the solution.